# Matrix Factorization on MovieLens 100K

Rating Accuracy, Retrieval, and Familiarity vs. Discovery

Student: Abaidullah Seikh Team: Individual
Email: abaidullahseikh06@gmail.com Date: 2026-10-13
Assignment: A05 Matrix Factorization (Week 5)

**Abstract:** HW5 asks whether a TensorFlow.js matrix-factorization recommender can improve held-out rating prediction while producing useful Top-10 recommendations rather than merely repeating past tastes. I implemented the six required functions on MovieLens 100K with a chronological per-user 80/10/10 split, 16-dimensional factors, validation checkpointing, Top-10 filtering, and Recall@10 evaluation. After diagnosing and correcting a collapsing L2 objective, the restored epoch-19 model achieved test RMSE 1.1165 versus a 1.2289 mean baseline, a 9.15% reduction, while Recall@10 was 0.0050. The user-level evidence therefore supports a mixed platform strategy: familiar recommendations can reduce choice friction, while a smaller discovery component can broaden catalog exposure, and neither benefit is established by RMSE alone.

**Index Terms:** recommender systems, matrix factorization, MovieLens, RMSE, retrieval

## 1. Introduction

**Problem statement.** HW5 has two linked tasks. The implementation task is to complete exactly six `TODO(hw5)` functions in `week5/script.js`: `initializeFactors`, `trainMF`, `predictRating`, `selectBestCheckpoint`, `recommendTopK`, and `recallAtK`: without changing the provided scaffolding. The mandatory analysis task is *Familiar Choices vs. New Discoveries*: compare two users' Top-10 lists with their past ratings and state what those lists are worth to a streaming platform [7].

**Motivation.** A recommender can lower pointwise rating error without producing a strong ranked list. The assignment therefore requires both RMSE and Recall@10, plus a user-level business interpretation; prior evaluation work likewise argues that predictive accuracy alone is not enough to describe recommender usefulness [3, 4].

**Concrete example.** In the corrected output, user 620 receives *Gumby: The Movie* and *Harriet the Spy*, which match previously observed Children's/Animation preferences, while user 421 receives *Microcosmos* (Documentary) and *The Big Green* (Children's), genres absent from that user's recorded history. The same model therefore produces a familiarity-heavy list for one user and a broader discovery mix for another.

**Contributions.**

- I implemented and verified the six required functions, including validation-selected checkpoint restoration, Top-10 filtering, and explicit Recall@10 exclusions.
- I traced a real training collapse to the original whole-matrix L2 term, fixed only that part of `trainMF()`, and used the corrected model to answer the mandatory familiarity-versus-discovery question with a platform decision based on catalog expansion rather than RMSE alone.

## 2. Related Work

MovieLens is a long-running recommendation research platform and its datasets are widely used for recommender-system evaluation; Harper and Konstan document their history, design, and limitations [1]. Matrix factorization is a standard collaborative-filtering approach in which low-dimensional user and item vectors model interaction structure [2]. McNee, Riedl, and Konstan argue that recommender evaluation should include user value beyond predictive accuracy [3]; Herlocker et al. likewise emphasize that recommender evaluation depends on the user task and should include properties beyond a single prediction-quality number [4]. TensorFlow.js provides browser-based tensor computation and automatic differentiation [5]; the implementation uses TensorFlow.js 4.22.0 tensor, optimizer, ranking, and memory-management APIs from the official API documentation [6]. I considered three alternatives explicitly: a mean-only predictor was retained only as the required non-personalized baseline; training RMSE was rejected as a checkpoint criterion because the assignment specifies validation RMSE and the last epoch need not generalize best; and, when the initial L2 formulation collapsed the factors, I compared normalized regularization alternatives under the same initialization instead of silently changing the optimizer or hyperparameters, selecting the batch-row formulation because it had the best validation RMSE in that controlled comparison.

## 3. Method

### 3.1 Approach and Pipeline

**Approach.** I used the supplied MovieLens 100K data and the provided chronological split instead of creating a new preprocessing pipeline. The model is a 16-factor matrix factorization trained in TensorFlow.js; model selection uses validation RMSE, while final evaluation reports test RMSE, Top-10 recommendations, and Recall@10. Rating prediction and ranked retrieval are treated as related but different tasks.

**Pipeline.**

1. Load `u.data` and `u.item`; keep missing ratings unobserved rather than filling them with zero.
2. Use the provided per-user timestamp ordering and 80/10/10 train/validation/test split.
3. Initialize user and movie factors and compute the training-set global mean.
4. Train in mini-batches with TensorFlow.js, record train/validation RMSE each epoch, and restore the best validation checkpoint.
5. Generate Top-10 lists after masking each user's training and validation movies.
6. Evaluate test RMSE, Recall@10 with exclusions, and the mandatory familiar-choice versus discovery question for two users.

### 3.2 Data and Model

MovieLens 100K contains 100,000 ratings from 943 users on 1,682 movies [1]. The provided data layer produced 79,619 training, 9,596 validation, and 10,785 test ratings with no split exclusions; the training mean was $\mu = 3.5806026199776437$. The model stores 16-dimensional user and movie factors and predicts

$$\hat{r}_{ui} = \text{clip}_{[1,5]}\left(\mu + p_u^\top q_i\right). \tag{1}$$

The factor matrices have shapes $944 \times 16$ and $1683 \times 16$ and are indexed by raw IDs. RMSE uses the clamped rating prediction, while Top-10 ranking uses the raw $\mu + p_u^\top q_i$ score, so a recommendation score slightly above 5 is a ranking score rather than a bounded star prediction.

### 3.3 Tools, Libraries, and Configuration

**Tools and libraries.**

- TensorFlow.js 4.22.0 in the browser, using the `webgl` backend and the official APIs `tf.variable`, `tf.randomNormal`, `tf.gather`, `tf.losses.meanSquaredError`, `tf.train.adam`, `optimizer.minimize`, `tf.topk`, and `tf.tidy` [6, 5].
- The supplied Week 5 HTML/JavaScript scaffold and MovieLens files from the course repository [7].
- OpenCode 1.18.31 with GPT-6 Astra (medium) for the coding/debugging session; its unmodified session export is submitted as the audit trail.

**Configuration.** The final run used 20 epochs, learning rate 0.01, L2 coefficient 0.05, batch size 64, and 16 latent factors. The controlled regularization comparison used one shared deterministic initialization; the final default run kept the scaffold's existing stochastic initializer, so I do not claim a multi-seed confidence interval.

### 3.4 Key Design Decisions

- **Validation checkpointing, not the last epoch.** Epoch 20 had lower training RMSE but worse validation RMSE than epoch 19, so the best validation snapshot was restored before test evaluation.
- **Regularize participating rows.** After measuring factor collapse under whole-matrix L2 applied at every mini-batch, I changed only the regularization expression to the batch-row mean of squared row norms:

$$L_B = \text{MSE}_B + \lambda \frac{1}{|B|} \sum_{(u,i)\in B} \left(\|p_u\|_2^2 + \|q_i\|_2^2\right). \tag{2}$$

- **Keep rating and retrieval evidence separate.** Test RMSE is compared with the training-mean baseline, while Recall@10 is reported independently with its exclusions; a lower RMSE is not treated as proof of a better ranked list [3, 4].

### 3.5 Recommendation and Recall Protocol

Top-10 scoring excludes movies a user rated in training or validation; test-rated movies remain eligible for ranking. Relevant retrieval items are test ratings $\geq 4$. The written README defines Recall candidates as movie IDs that appear anywhere in the training split, so training-pool membership determines the relevant denominator and the unseen-item exclusions [7]. The supplied starter code then instructs `recallAtK` to call `recommendTopK(userId,K)`, while the `recommendTopK` contract ranks all eligible catalog movies after masking that user's training and validation items. The provided rank-1/rank-11 self-check makes this difference observable: its fixture has only three training-pool IDs, so a strictly training-only Top-10 could not place the relevant item outside the Top-10, yet the check requires Recall@10 to fall to zero when catalog competitors move that item to rank 11. I therefore report the executable protocol exactly rather than relabeling it as strict training-only ranking: training-pool membership defines relevant eligibility and exclusions, and `recommendTopK` supplies the ranking seen by the supplied checks. For the two users reported in the business analysis, all 20 returned Top-10 movies also belonged to the global training pool, and none overlapped that user's training or validation history.

## 4. Experiments

### 4.1 Setup and Results

The untouched starter produced 4 passed / 0 failed / 7 pending checks. After implementation and the training fix, the browser reported 11 passed / 0 failed / 0 pending. The corrected 20-epoch run selected epoch 19; epoch 20 had lower training error but worse validation error, so checkpoint restoration was genuinely exercised. The corrected model reduces test RMSE by 0.112415, or 9.1476%, relative to the training-mean baseline, while Recall@10 is only 0.5005%. Before the L2 fix the degenerate model had Recall@10 2.1137%; retrieval therefore worsened by about 76.3% even while pointwise rating error improved, and I report both outcomes because selecting only the positive RMSE result would hide an important failure mode.

### 4.2 Learning Trajectory and Checkpoint Selection

Table 2 shows the complete validation trajectory from the corrected default run. Training RMSE generally decreases, but validation RMSE oscillates rather than improving monotonically. The minimum occurs at epoch 19 (1.071445); epoch 20 lowers train RMSE further to 0.765262 but worsens validation RMSE to 1.086943. This is precisely why the validation checkpoint is operationally important: reporting the last epoch would have produced a worse held-out model than restoring epoch 19.

### 4.3 Verification: Sanity Checks and Manual Checks

Several checks were used for c2 rather than relying on the final table alone. A toy baseline with ratings 2 and 4 has mean 3 and hand-checkable RMSE

$$\sqrt{\frac{(3-2)^2 + (3-4)^2}{2}} = 1,$$

which matched the page. A prediction fixture gives raw $3 + 1 \times 3 + 2 \times 4 = 14$ and correctly clamps to 5. The restored checkpoint's validation RMSE exactly matched the selected epoch value, and an independent JavaScript-number computation gave validation/test RMSE 1.0714449069/1.1164853004 versus tensor-helper values 1.0714447826/1.1164852683 (about $1.3 \times 10^{-7}$ disagreement). For a

Table 1: Corrected Model and Retrieval Results

| Measure | Value |
|---|---|
| Final self-checks | 11 / 0 / 0 |
| Backend | WebGL |
| Selected epoch | 19 |
| Train RMSE (epoch 19) | 0.769141 |
| Validation RMSE | 1.071445 |
| Restored validation RMSE | 1.071445 |
| Mean-baseline test RMSE | 1.228900 |
| MF test RMSE | 1.116485 |
| Relative RMSE reduction | 9.1476% |
| Recall@10 | 0.005005 |
| Evaluable users | 861 |
| Unseen relevant movie IDs | 13 |
| Users without eligible relevant item | 82 |

Table 2: Corrected 20-Epoch Training History

| Ep. | Train | Val. | Ep. | Train | Val. |
|---|---|---|---|---|---|
| 1 | 1.023202 | 1.097195 | 11 | 0.780605 | 1.082685 |
| 2 | 0.959732 | 1.122323 | 12 | 0.783319 | 1.106884 |
| 3 | 0.879643 | 1.084867 | 13 | 0.777122 | 1.081796 |
| 4 | 0.840354 | 1.077567 | 14 | 0.775797 | 1.100728 |
| 5 | 0.818438 | 1.076261 | 15 | 0.773541 | 1.081658 |
| 6 | 0.814386 | 1.098126 | 16 | 0.770129 | 1.096235 |
| 7 | 0.797764 | 1.082559 | 17 | 0.771116 | 1.075516 |
| 8 | 0.797345 | 1.102057 | 18 | 0.767745 | 1.091922 |
| 9 | 0.789510 | 1.085031 | **19** | **0.769141** | **1.071445** |
| 10 | 0.789925 | 1.105379 | 20 | 0.765262 | 1.086943 |

real retrieval check, user 70 had two eligible relevant test movies, IDs 217 and 408; the Top-10 contained 408 at rank 7 but not 217, so Recall@$10_{70} = 1/2 = 0.5$, exactly matching the implementation. Train/validation overlap was also empty for the recommendation lists used below, and a second Recall@10 evaluation reproduced the complete result, including the 13 unseen relevant IDs and 82 users without eligible relevant items.

Table 3: Independent and Cross-Check Evidence

| Check | Observed result |
|---|---|
| Starter/final harness | 4/0/7 before implementation; 11/0/0 after final fix |
| Toy mean-baseline RMSE | Hand-computed 1.0; page returned 1.0 |
| Prediction fixture | Raw score 14; clamp returned 5 |
| Checkpoint restoration | Active validation RMSE exactly 1.0714447826, equal to selected epoch 19 |
| Independent test RMSE | Tensor helper 1.1164852683 vs. separate JS-number calculation 1.1164853004 |
| Top-10 exclusion | No train/validation overlap for users 620, 421, or 843 |
| Manual Recall@10 | User 70: one hit among two eligible relevant movies, $1/2 = 0.5$ |
| Repeat retrieval evaluation | Overall Recall@10 and exclusion sets reproduced exactly |
| File integrity | Protected files unchanged; Step 2C changed only the L2 expression/comments inside `trainMF()` |

## 5. Discussion

### 5.1 Failure Case

The first apparently complete implementation passed 11/0/0 self-checks, yet every one of its 20 train/validation RMSE rows was effectively identical and recommendation scores collapsed to the training mean. Its test RMSE was 1.2288999214, only about $2.16 \times 10^{-7}$ below the 1.2289001372 baseline. Passing the harness therefore did not prove that the learning objective was behaving sensibly.

### 5.2 Root Cause

The initial objective applied $\lambda(\sum P^2 + \sum Q^2)$ to the complete factor matrices at every mini-batch. On the same initial 64-rating batch, MSE was 1.4140 while the weighted L2 term was 5.2954 (3.745× the MSE); aggregate L2-gradient norms were about 9.1× and 17.0× the corresponding user/item MSE-gradient norms, and factor norms fell from 6.20/8.21 initially to about $1.10 \times 10^{-27}$/$1.38 \times 10^{-27}$ after one epoch.

### 5.3 Fix and Verification

I compared four formulations for three epochs using the same seeded initial factor arrays; the original formulation remained degenerate, whereas three normalized alternatives produced non-identical scores, and batch-row mean regularization achieved the best validation RMSE among those alternatives (1.0780 at epoch 3). I therefore changed only the L2 expression inside `trainMF()`; after the change, the browser still passed 11/0/0, recommendation scores varied, checkpoint restoration remained exact, and test RMSE improved to 1.1165. This is the report's BUILD → ASK WHY → TEST → EXPLAIN → IMPROVE loop.

Table 4: Controlled Three-Epoch Regularization Comparison

| Var. | Regularization scope | Train RMSE | Val. RMSE | Distinct Top-10 scores |
|---|---|---|---|---|
| A | Whole matrices, unnormalized sums | 1.1052 | 1.1830 | 1 |
| B | Batch rows, mean squared row norms | 0.8722 | **1.0780** | 10 |
| C | Whole matrices, element means | 0.8323 | 1.1118 | 10 |
| D | Batch rows, element means | 0.8292 | 1.1161 | 10 |

Variant B was chosen because it fixed the measured collapse while changing only the regularization scale/scope and gave the best

validation RMSE among the controlled alternatives. This comparison was not used as a test-set model-selection sweep: the alternatives were compared on validation behavior under the same seeded initialization, and the final reported test RMSE comes only from the subsequently trained corrected default model.

### 5.4 What Worked

- Keeping the fix narrow worked well: only the L2 expression/comments inside `trainMF()` changed in the final correction, while the optimizer, batching, checkpoint logic, the other five student functions, and professor scaffolding stayed unchanged.
- Independent number checks were useful. The hand-computed toy RMSE, the separate JavaScript-number RMSE calculation, and the manual user-70 Recall@10 check all agreed with the page output.
- Restoring by validation RMSE mattered in practice: epoch 19 was better on validation than epoch 20 even though training error continued to decrease.

### 5.5 What Surprised Me or Did Not Work

- The first implementation passed 11/0/0 but had effectively collapsed factors and mean-like predictions. Passing the harness alone was not enough evidence that training was sensible.
- After the L2 fix, test RMSE improved by 9.15%, but Recall@10 dropped from 0.02114 to 0.00501. Better pointwise rating prediction did not translate into better Top-10 retrieval.
- User 843 was a clear mismatch case: 18 of 30 highly rated historical movies were Sci-Fi, but the corrected Top-10 had no Sci-Fi and eight Comedy labels. Non-degenerate scores still did not guarantee preference alignment.

### 5.6 Mandatory Question: Familiar Choices vs. New Discoveries

I chose users 620 and 421 because their corrected lists show two different recommendation roles and neither Top-10 overlaps the user's training or validation ratings. User 620 had 99 historical train/validation ratings, including 64 rated at least 4; Children's appeared 35 times and Animation 22 times, with highly rated examples such as *Jumanji*, *E.T.*, and *The Santa Clause*. Every genre in the new Top-10 had already appeared in the user's history and among highly rated movies, so *Gumby: The Movie* and *Harriet the Spy* are mainly familiar choices. User 421 had 55 historical ratings and 39 rated at least 4; Thriller (24), Action (17), Drama (16), and Crime (13) were prominent, with highly rated examples including *Die Hard*, *The Rock*, and *GoodFellas*. *True Crime* and *Ben-Hur* keep familiar cues, but Documentary and Children's were absent from both the full recorded history and the highly rated subset, so *Microcosmos* and *The Big Green* are genre-level discovery candidates in this data. **Business axis and platform decision.** I use *catalog-expansion opportunity* as the decision axis: how much the Top-10 broadens exposure beyond the user's recorded preferences while still retaining familiar anchors. On that axis, user 620 has **0 new genre categories** in the Top-10, so I would use this list as a familiarity slate rather than claim it creates discovery value. User 421 has **2 previously unseen genre categories**: Documentary and Children's, while still retaining familiar Action/Thriller/Drama cues, so I would use this list as the stronger mixed exploration slate. The decision therefore turns on a product quantity that RMSE and Recall@10 do not measure: observed breadth of catalog exposure relative to each user's history. For a streaming platform, the practical action is to preserve familiar anchors for confidence but allocate discovery slots when the list demonstrably expands the user's catalog exposure; in these two cases that means treating user 620 as familiarity-first and user 421 as exploration-capable. This is an offline product decision, not a causal claim: I do not infer more starts, completion, engagement, or retention from the genre comparison. Those outcomes would require online evidence. As a next offline check I would report relevance and discovery side by side, for example Recall@10 together with catalog coverage or a novelty measure, consistent with evaluation work that separates recommendation tasks and non-accuracy properties [4].

Table 5: User 620 Top-10: Familiar-Genre Reinforcement

| R | Movie | Score | Genre(s) |
|---|---|---|---|
| 1 | What Happened Was... | 5.275 | Comedy/Drama/Rom. |
| 2 | Dream With the Fishes | 5.222 | Drama |
| 3 | Gumby: The Movie | 5.175 | Animation/Children's |
| 4 | North | 5.167 | Comedy |
| 5 | Lord of Illusions | 5.134 | Horror |
| 6 | Love in the Afternoon | 5.132 | Comedy/Romance |
| 7 | Harriet the Spy | 5.101 | Children's/Comedy |
| 8 | The Designated Mourner | 5.074 | Drama |
| 9 | I Love Trouble | 5.032 | Action/Comedy |
| 10 | Boxing Helena | 4.997 | Mystery/Rom./Thr. |

Table 6: User 421 Top-10: Familiar Anchors Plus Discovery

| R | Movie | Score | Genre(s) |
|---|---|---|---|
| 1 | True Crime | 5.384 | Mystery/Thriller |
| 2 | Ben-Hur | 5.236 | Action/Adv./Drama |
| 3 | Microcosmos | 5.195 | Documentary |
| 4 | The Big Green | 5.173 | Children's/Comedy |
| 5 | Best of the Best 3 | 5.029 | Action |
| 6 | Even Cowgirls Get the Blues | 4.935 | Comedy/Romance |
| 7 | Bitter Moon | 4.934 | Drama |
| 8 | Stuart Saves His Family | 4.930 | Comedy |
| 9 | A Modern Affair | 4.920 | Romance |
| 10 | I'll Do Anything | 4.916 | Comedy/Drama |

### 5.7 Next Improvement

With more time, my first improvement would be to select or train the model with a retrieval-aware validation target instead of relying only on squared rating error, then repeat the comparison across several fixed random seeds. This follows directly from the observed result: test RMSE improved by 9.15% while Recall@10 fell by 76.3%. I would keep RMSE for rating quality, but add a retrieval metric and a simple novelty/coverage measure to the validation dashboard so the chosen model is not good at one objective and poor at the other. The current run is also limited by one final stochastic initialization and by using genre presence as a simple discovery proxy rather than a direct measure of user satisfaction.

## 6. AI Usage Disclosure

**AI tools used.**

- OpenCode 1.18.31 with OpenAI GPT-6 Astra (medium).

How AI was used. OpenCode was used for starter inspection, code generation inside the six required stubs, browser execution, debugging, documentation lookup, controlled regularization comparisons, and verification. TensorFlow.js 4.22.0 was the implementation library, not an AI tool.

**What I personally verified.** I checked the arithmetic shown in the report, including the toy RMSE of 1 and the user-70 Recall@10 calculation of $1/2 = 0.5$. I also checked that the reported recommendation titles/scores match the recorded session output and that the RMSE improvement calculation uses the recorded baseline and corrected test values. The OpenCode session itself contains the executed browser evidence for 4/0/7 before implementation, 11/0/0 after the final fix, epoch-19 checkpoint restoration, the full 20-epoch run, the independent JavaScript-number RMSE cross-check, repeated Recall@10 evaluation, train/validation overlap checks, tensor-count check, and protected-file hashes.

**What I did not independently re-run.** I did not run a second independent 20-epoch training job outside the submitted OpenCode session, and I do not claim that I did. I also do not treat any proposed experiment that was not executed as evidence. This distinction is important because the course rubric requires claimed verification to match the audit trail.

**Session log reference.** The submitted `seikh_abaidullah_a05_session.json` is the unmodified tool-native export for session `ses_ece7abc1dffe0mqjLY7cP4hcRa`. It records the prompts, tool calls, file operations, and experiment milestones used for this assignment.

Table 7: Verification Evidence Used in the Report

| Evidence | How it was checked |
|---|---|
| Toy RMSE = 1 | Arithmetic is shown in §4 and agrees with the browser result. |
| User-70 Recall@10 = 0.5 | Two relevant candidate movies were counted manually; one appears in the Top-10. |
| Final self-checks | Browser log in the OpenCode session reports 11 passed, 0 failed, 0 pending. |
| Checkpoint restoration | Selected and active validation RMSE are both 1.0714447826 at epoch 19. |
| Test RMSE | Tensor helper 1.1164852683 agrees with a separate JS-number calculation 1.1164853004. |
| Recommendation exclusions | Train/validation intersections were checked for users 620, 421, and 843. |
| Recall exclusions | Repeated evaluation reproduced 13 unseen relevant IDs and 82 users without an eligible relevant item. |
| File integrity | Protected files were hash-checked; the final correction changed only the intended L2 expression/comments inside `trainMF()`. |
| References | Metadata and URLs were checked against publisher, GroupLens, Google Research, TensorFlow.js, and the official Week 5 repository. |

## References


[1] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, Art. 19, pp. 1–19, Dec. 2015, doi: 10.1145/2827872. [Online]. Available: `https://doi.org/10.1145/2827872`. [Accessed: 2026-10-13].

[2] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009, doi: 10.1109/MC.2009.263. [Online]. Available: `https://doi.org/10.1109/MC.2009.263`. [Accessed: 2026-10-13].

[3] S. M. McNee, J. Riedl, and J. A. Konstan, "Being Accurate Is Not Enough: How Accuracy Metrics Have Hurt Recommender Systems," in *CHI '06 Extended Abstracts on Human Factors in Computing Systems*, Montreal, QC, Canada, 2006, pp. 1097–1101, doi: 10.1145/1125451.1125659. [Online]. Available: `https://doi.org/10.1145/1125451.1125659`. [Accessed: 2026-10-13].

[4] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl, "Evaluating Collaborative Filtering Recommender Systems," *ACM Trans. Inf. Syst.*, vol. 22, no. 1, pp. 5–53, Jan. 2004, doi: 10.1145/963770.963772. [Online]. Available: `https://doi.org/10.1145/963770.963772`. [Accessed: 2026-10-13].

[5] D. Smilkov *et al.*, "TensorFlow.js: Machine Learning for the Web and Beyond," in *Proc. SysML*, Palo Alto, CA, USA, 2019. [Online]. Available: `https://research.google/pubs/tensorflowjs-machine-learning-for-the-web-and-beyond/`. [Accessed: 2026-10-13].

[6] TensorFlow.js, "TensorFlow.js API Reference." [Online]. Available: `https://js.tensorflow.org/api/latest/`. [Accessed: 2026-10-13].

[7] S. Jin, "RecSys-LLMs: Week 5 - From Ratings to Recommendations (Matrix Factorization)," GitHub repository, 2026. [Online]. Available: `https://github.com/dryjins/RecSys-LLMs/tree/main/week5`. [Accessed: 2026-10-13].